RESEARCH REPORT

Binary consistency machines

Comparison and a search below 196 states

13 November 2026

Outcome. No verified binary blank-tape machine with fewer than 196 working states was obtained. The requested construction therefore remains unfinished. This report records exact comparisons, newly checked failed reductions, a mathematical obstruction to one logical simplification, and reproducible artifacts. It does not present a rejected 195-state table as a solution.

Requested target

$$\mathrm{PA} \vdash \Big[(\text{the proposed machine never halts from blank}) \iff \mathrm{Con}(\mathrm{PA})\Big], \qquad |Q| < 196.$$

Sources compared. The designated 208-state constructions in Vladimir Reshetnikov's ProveIt incoming material; James Chang's 209-state predecessor at the specified commit; and the 196-state BusyBeaver machine at a pinned current commit.

Reading guide. Sections 1–4 give the comparison and logical architecture. Sections 5–6 separate the internal consistency argument from external soundness and prove a barrier to a tempting simplification. Sections 7–9 document the search, its limits, and reproducibility. The accompanying archive contains the literal tables, TeX source, proof notes, Python and C++ code, and machine-readable receipts.

Prepared with parallel computational and mathematical audits. Numerical searches are explicitly bounded. Previously supplied research and proof logs are identified as inherited material.

Contents

1 Scope, conventions, and evidential status **2**

2 Pinned inputs and exact comparison **2**

2.1 Source pins . . . 2

2.2 Nine 208-state tables, two actual machines . . . 3

2.3 Cross-lineage comparison . . . 3

3 Where the 196 states go **4**

4 The logical compression in PA196 **5**

4.1 Exact constructors and aliases . . . 5

4.2 The guarded gate . . . 5

5 The internal PA equivalence: published proof and remaining bridges **6**

5.1 Published endpoint versus external soundness . . . 6

5.2 A syntactic route to the converse . . . 6

5.3 Commands, source code, and positive tape progress . . . 7

6 A proved obstruction: equality reflexivity cannot replace the witness **7**

6.1 Why the first head also needs care . . . 8

7 New computational searches and their limits **9**

7.1 Reassociation and the invalid 195-state merge . . . 9

7.2 The return scanner . . . 9

7.3 Engine folds and reachable source contexts . . . 10

7.4 Logical and grammar searches . . . 10

7.5 Register reassignment: a new 196-state control example . . . 11

8 What ConciseIPRA contributes **12**

9 Reproduction, artifacts, and remaining work **12**

9.1 Archive layout . . . 12

9.2 What would complete the requested result . . . 13

1 Scope, conventions, and evidential status

A machine here has one two-way infinite tape, alphabet $\{0, 1\}$, blank symbol 0, a distinguished starting working state, and deterministic instructions

$$(q, b) \longmapsto (w, d, q'), \qquad d \in \{L, R\}.$$

The reported state count excludes the halt state. All working-state rows are present. No larger alphabet, nonblank advice, stay-put instruction, or change of state-count convention is used to obtain a smaller number.

Write H_T for the ordinary arithmetical assertion that the exact literal machine T halts from the blank tape. The target is a machine with fewer than 196 working states together with a PA proof of

$$\neg\mathsf{H}_T \leftrightarrow \mathrm{Con(PA)}.$$

A finite simulation of a long initial segment is insufficient. Likewise, a shorter source program, a smaller grammar, or identical rows in an altered table do not validate the preceding source rewrite. Quotienting genuinely identical rows is valid; it preserves whatever behavior that altered table already has.

There are four distinct kinds of evidence in this report:

1. **Exact finite identities:** literal table hashes, rooted transition-graph isomorphisms, resolved source control flow, and finite table differences.
2. **Symbolic runtime checks:** complete primitive frames with arbitrary unary register values and an affine tape gap, using the existing proved register excursions as oracles.
3. **Written mathematical arguments:** the consistency-strength analysis and proof-translation construction. These are not new Lean declarations.
4. **Bounded searches:** exhaustive enumeration within stated windows, step bounds, helper models, or candidate families. These are not global lower bounds.

The exact comparison was rerun. The standard PA196 symbolic checker was also rerun against all 4,959 primitive branch contracts. New candidate checks and search receipts are included. A previous incoming archive contains a Lean rebuild of the physical endpoint; that build is inherited evidence, not a fresh end-to-end rebuild performed for this report. No new fully formalized sub-196 correctness theorem is claimed.

2 Pinned inputs and exact comparison

2.1 Source pins

The comparison uses the following fixed revisions; branch names alone are not reproducible references.

ProveIt
`f7d2482e35cf2f8f3bd03b5a82387ad999243548`.
BusyBeaver, 196 states
`407cdc356f6b586635e71ac92be394255f4bfea4`.
BusyBeaver, requested 209 states
`69d98241a721ead20b4f0b17df9062950b40fbfe`.

All twelve Con(PA)-related ZIP files listed in the inspected incoming directory were retrieved. Five designated construction packages contain nine final 208-state tables. Other packages contain comparison reports, earlier ancestors, or unsuccessful searches; they are not additional successful sub-208 constructions. In particular, the incoming `pa_tm_research_2026-11-13.zip` is an

earlier unsuccessful attempt at this same below-196 problem. Its useful source snapshots and checker infrastructure are reused under `support/`, with their original attribution.

The exact 196-state literal table has SHA-256

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8.`

Its literal working states are 0–195, its start is 0, and its halt label is 196. Its sole transition to halt is

$$(63, 1) \longmapsto (1, R, 196).$$

Sparse native names in the source are different: the start is 74, the program root is 84, and the halt name is 254. Mixing these numbering systems would invalidate a proposed patch.

2.2 Nine 208-state tables, two actual machines

Canonicalizing the rooted action-labelled transition graph gives precisely two isomorphism classes among the nine final tables. The canonicalization preserves scanned symbol, written symbol, movement direction, start state, and halt. It does not identify unrelated machines merely because their state counts agree.

Construction package	Class A	Class B
`U208_ConPA`	1	0
`U208_ConPA_Conditional_Context`	1	1
`U208_ConPA_Entry_Context`	1	1
`U208_ConPA_History`	1	1
`u208_research`	1	1
Total	5	4

The classes have canonical SHA-256 hashes beginning `5d858b5d7528dd98` and `f0c6098fabd68fce3`. Full hashes and explicit state maps are in `comparison/fresh_comparison.json`. Multiple literal hashes within a class reflect renumbering or serialization, not an additional independent saving.

Both 208-state classes execute the same flattened sequence of 3,909 primitive occurrences and have the same resolved ordinal control-flow graph. Their tree associations and decoder layouts differ. Their final one-state saving is the same contextual identification of native roles 172 and 211. Keeping one name or the other does not create two independently composable reductions. The four local simulation paths for the two roles and two read symbols have lengths 7, 1, 3, and 1.

2.3 Cross-lineage comparison

Design	States	Rows	Occurrences	Contracts	Max. path
U208, class A	208	416	3,909	5,084	42
U208, class B	208	416	3,909	5,084	41
Requested PA209	209	418	3,948	5,135	33
Current PA196	196	392	3,812	4,959	38

A contract is counted for each increment, jump, or halt occurrence and for each of the two outcomes of a decrement. All four source trees have maximum consecutive-zero path length six. Their internal source DAG node counts are respectively 174, 174, 173, and 159.

The requested James Chang PA209 is *not* the U209 parent packaged with the U208 reductions. This matters: replacing the number 209 by 208 in an informal lineage diagram would conflate different source programs.

The native-label PA209 and PA196 tables have 134 working-state names in common. Of their 268 common rows, 265 are identical. Only these three common rows change:

Native row	PA209 action	PA196 action
101/1	1 R 115	1 R 1367
164/0	0 L 311	0 R 77
190/1	1 L 308	1 R 99

The replacement removes 75 old controls and introduces 62:

$$209 - 75 + 62 = 196.$$

Both native roles 172 and 211 have already disappeared. Therefore the final U208 contextual fold supplies no extra state deletion in PA196.

3 Where the 196 states go

The current implementation decomposes into 36 engine states, 13 leaf routers, and 147 physical readers for a 159-node source DAG:

$$196 = 36 + 13 + 147.$$

The axiom-generation word has 201 macro items and 68 distinct internal AX nodes. Six contextual folds implement those nodes with 62 states. The fixed remainder costs 134 states. In this architecture a candidate must therefore achieve a net AX cost at most 61, or save a state in the remainder, without breaking startup or the runtime invariant.

A source tree is encoded by a path of tape bits. The decoder follows this path to an increment, decrement, jump, or halt primitive. The same decoder control may represent several source roles when a preceding path bit disambiguates them. A left-moving excursion recovers that bit and returns to the current position. This is more powerful than ordinary identical-subtree sharing, but makes re-association delicate.

The arithmetic macros manipulate a stack of numerical formula codes using Cantor pairing. The engine represents registers by unary blocks. Its primitive simulation checks arbitrary register values; testing only a few small concrete register files would miss failures involving the tape gap, marker, or return path.

The exact blank initializer reaches the clean program entry after 58,154 literal transitions. In native coordinates it has head -2 at root 84, marker ones at $-3, -2$, header 46, and physical register prefix

$$(0, 0, 0, 0, 0, 0, 2, 0, 0),$$

followed by 26 one-valued spare registers. Startup is a separate obligation. A change that preserves an abstract register projection after initialization can nevertheless destroy the blank initializer.

4 The logical compression in PA196

4.1 Exact constructors and aliases

Let

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b.$$

The numerical constructors are

$$\mathsf{I}(a,b) = 4p(a,b), \quad \mathsf{M}(a,b) = 4p(a,b)+1, \quad \mathsf{U}(a,B) = 4p(B,a)+2, \quad \mathsf{E}(a,b) = 4p(a,b)+3.$$

The universal constructor is transposed: its label is the right Cantor component. Write

$$\mathsf{N}(B) = \mathsf{I}(B,1), \qquad B \wedge C = \mathsf{N}(\mathsf{I}(B,\mathsf{N}(C))), \qquad \exists_a B = \mathsf{N}(\mathsf{U}(a,\mathsf{N}(B))).$$

Zero is decoded as truth. There is a real alias $\mathsf{I}(0,0) = 0$; it must be handled explicitly in proof translations. The distinguished contradiction code is $1 = \mathsf{M}(0,0)$.

In the AX command put $D = d + x$, $H = h + y$, and $A = c$. The command pushes nine items: its saved top, a generalization $\mathsf{U}(d,h)$, the restricted propositional head

$$((H \to d) \to D) \to ((D \to H) \to (A \to H)), \qquad d \le D,$$

and the six following heads. The guarded gate is left in the accumulator, rather than pushed as a tenth stack item.

$$\begin{aligned}
h_1 &= (\exists_D \mathsf{U}(D,H)) \to H,\\
h_2 &= \mathsf{U}(D,\mathsf{U}(A,H)) \to \mathsf{U}(A,\mathsf{U}(D,H)),\\
h_3 &= \mathsf{M}(H,D) \to (\mathsf{E}(H,A) \to \mathsf{M}(A,D)),\\
h_4 &= \mathsf{E}(H,D) \to (\mathsf{M}(A,H) \to \mathsf{M}(A,D)),\\
h_5 &= \exists_D \mathsf{E}(H,D),\\
h_6 &= \mathsf{U}(D,A) \to (\mathsf{U}(D,\mathsf{I}(A,H)) \to \mathsf{U}(D,H)).
\end{aligned}$$

Here the same natural number can serve as a variable label in an atom and as a formula code elsewhere. This deliberate coding feature is essential to the compact gate.

4.2 The guarded gate

For $c \neq a, b$, define

$$\begin{aligned}
\Delta &= \mathsf{E}(a,b) \to \mathsf{U}(c,\mathsf{E}(a,b)),\\
V &= \mathsf{M}(a,b) \to \mathsf{U}(c,\mathsf{M}(a,b)),\\
P &= c \to \mathsf{M}(c,b),\\
S &= \mathsf{U}\big(b,(\mathsf{M}(a,b) \to P) \wedge (\mathsf{M}(c,b) \to c)\big),\\
Q &= \mathsf{U}(c, S \to \mathsf{N}(P)),\\
F(a,b,c) &= \mathsf{N}\big(\Delta \to (V \to Q)\big).
\end{aligned}$$

When the guard fails, the command returns zero. Under classical reasoning the negative bundle supplies equality vacuity, membership vacuity, and a pointed separation principle. Every variable label in the decoded formula c is smaller than c. Consequently the variable with label c is fresh in that formula, even though c also names the property being separated.

The PA209-to-PA196 change transposes the universal constructor, drops the old explicit equality-transitivity head, and adds equality vacuity to the guarded bundle. It also changes head order and implication orientation where logically permitted and avoids two older auxiliary controls. This is a logical and compilation change, not just removal of syntactically duplicate table rows.

5 The internal PA equivalence: published proof and remaining bridges

5.1 Published endpoint versus external soundness

The pinned `PA196/Independence.lean` exposes the internal implication

$$\mathrm{PA} \vdash \neg \mathsf{H}_{196} \to \mathrm{Con}(\mathrm{PA}).$$

The repository also proves external nonhalting through a hereditarily finite model and a physical simulation. An external Lean theorem of nonhalting is not, merely by being formalized, the converse implication inside PA.

A detailed written proof-translation construction is supplied in `proof_notes/internal_reverse.md`. It explains how the converse can be arithmetized and identifies its remaining formal implementation obligations. This report does not describe that note as a newly compiled Lean theorem of the biconditional.

5.2 A syntactic route to the converse

Interpret a set code as a natural number whose one-bits encode its members. In a conservative definitional extension of PA with exponentiation, use

$$\mathrm{Bit}(A, b) \equiv \exists q \exists r\, [r < 2^b \ \wedge \ A = q2^{b+1} + 2^b + r].$$

Translate source variable x_i to arithmetic variable v_{2i} and reserve odd indices for local binders. Translate membership to Bit, equality to equality, implication to implication, and $\mathsf{U}(a, B)$ to $\forall v_{2a} J(B)$.

The zero alias requires a fixed equivalence proof between the translation of $\mathsf{I}(0, 0)$ and $J(0) \to J(0)$. The translation of code 1 is $\mathrm{Bit}(v_0, v_0)$, not a literal falsity symbol. PA proves

$$\mathrm{Bit}(A, b) \to b < A,$$

and hence proves $\forall u \neg \mathrm{Bit}(u, u)$. This supplies the bridge for every occurrence of numerical negation.

For every inserted arithmetic formula $\psi(i, \bar{z})$, PA proves finite bit comprehension:

$$\forall n \ \exists s < 2^n \ \forall i < n \, \big(\mathrm{Bit}(s, i) \leftrightarrow \psi(i, \bar{z})\big). \tag{FC}$$

Use this displayed existential statement as the induction invariant. At zero take $s = 0$. At the successor split on the formula $\psi(n, \bar{z})$, and choose respectively $s + 2^n$ or s. The lower bits are unchanged and the new bit has the required value. This is a proof template using an ordinary PA induction instance, not an algorithm deciding the truth of ψ.

Substitute

$$\psi(i) = \big(\mathrm{Bit}(A, i) \vee i = B\big) \wedge \varphi(i), \qquad n = A + B + 1.$$

This gives a finite witness for the gate's pointed separation clause. The syntactic self-freshness of property code c makes its variable available as the witness binder. If the labels $a = b$ coincide, the first antecedent under the b-binder becomes self-membership, so that branch is discharged by the PA theorem forbidding self-membership. This case cannot be ignored as an informal alpha-renaming detail.

The six heads have uniform ordinary logical proof templates. The restricted propositional head is a Boolean tautology for all values, so its numerical restriction is unnecessary for soundness.

Numeric modus ponens and closed generalization compile to their PA counterparts, with the zero and contradiction bridges inserted.

The intended primitive recursive compiler therefore has the syntactic correctness specification

$$\mathrm{NativeProof}(p, a) \to \mathrm{PAProof}(\mathrm{compile}(p), J(a)). \tag{PC}$$

At $a = 1$, append the no-self-membership theorem to obtain a PA refutation. The assertion concerns verification of the generated proof string. It is not a uniform truth predicate or a reflection principle for PA.

5.3 Commands, source code, and positive tape progress

Annotate every formula in the command store and stack by its compiled PA proof. AX creates proofs by the templates above; MP combines them or returns truth; Shift detects a sum equal to one only when one available code is the contradiction code. Primitive recursion on a finite command trace gives a corresponding PA refutation.

To connect this argument to the literal machine, the fixed startup and the source/tape contracts must also be arithmetized. They use finite lists, natural-number counters, elementary pairing operations, and induction on unary blocks or finite source macros. The relevant physical endpoint is `blank_halting_iff_source`; positive runtime segments prevent a successful source computation from silently using zero tape steps.

The arithmetic progress argument is simple but necessary. Suppose PA proves

$$\mathrm{Inv}(c) \to \exists n > 0, d\, [\mathrm{Run}(c, n, d) \wedge \mathrm{Inv}(d)].$$

If such a configuration halted after m steps, use strong induction on m. The next positive segment either has length at most m, yielding a smaller halting time from another invariant configuration, or crosses the alleged halt, contradicting that it is a successful nonterminal segment. Determinism and the exact halting convention justify both cases.

This gives a written route to

$$\mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA}) \to \neg \mathsf{H}_{196}.$$

The written strategy does not yet supply every required internal bridge. A complete implementation must expose the compiler, prove its syntactic specification, connect the padded/beta-coded literal trace formula to the invariant argument, and target Foundation's exact falsum and consistency predicate. Standard-natural-number truth equivalence in the existing sentence-correctness theorem is insufficient by itself. These are proof obligations, not merely a request to rerun Lean. For a smaller machine one also needs a PA-formalizable equivalence of its halting sentence with H_{196}, or a fresh direct proof of both consistency implications.

6 A proved obstruction: equality reflexivity cannot replace the witness

A tempting shortening replaces

$$h_5 = \exists_D \mathsf{E}(H, D) \quad \text{by} \quad \mathsf{E}(H, H),$$

leaving every other axiom and rule unchanged. This replacement is not adequate for the requested consistency strength.

Proposition 1. *Let S_{refl} be the resulting numerical calculus, with the same guarded gate, restricted propositional head, modus ponens, and closed generalization. PA proves $\mathrm{Con}(S_{\mathrm{refl}})$. If PA is consistent, PA cannot prove $\mathrm{Con}(S_{\mathrm{refl}}) \leftrightarrow \mathrm{Con}(\mathrm{PA})$.*

Proof. Define a Boolean valuation j on the exact numerical codes by

$$\begin{aligned} j(0) &= \top, \\ j(\mathsf{I}(a,b)) &= j(a) \Rightarrow j(b), \\ j(\mathsf{M}(a,b)) &= \begin{cases} \bot, & a = b, \\ j(a), & a \neq b, \end{cases} \\ j(\mathsf{U}(a,b)) &= j(b), \\ j(\mathsf{E}(a,b)) &= (a = b). \end{aligned}$$

For nonzero compound codes, every recursively used argument is smaller than the code. Course-of-values recursion, implemented by a coded finite Boolean list, makes j primitive recursive and PA-provably total. At the only recursive exception $\mathsf{I}(0,0) = 0$, the implication identity remains valid because $\top \Rightarrow \top = \top$.

In particular, $j(1) = j(\mathsf{M}(0,0)) = \bot$. Thus $j(\mathsf{N}(B)) = \neg j(B)$ and $j(\exists_a B) = j(B)$.

Head 1 becomes $j(H) \Rightarrow j(H)$. Head 2 has equal antecedent and consequent values. In head 3, a true equality antecedent means $H = A$, making the membership codes identical; otherwise that antecedent is false. The same argument with $H = D$ proves head 4. The replacement head is true. Head 6 is

$$j(A) \Rightarrow ((j(A) \Rightarrow j(H)) \Rightarrow j(H)),$$

a tautology. The propositional head is also a tautology. Modus ponens and generalization preserve truth under j.

For the gate, the guard implies $c \neq b$, so $j(\mathsf{M}(c,b)) = j(c)$. Consequently $j(P) = \top$, $j(\mathsf{M}(c,b) \to c) = \top$, and $j(S) = \top$. It follows that $j(Q) = \bot$. Both Δ and V are true because quantification is the identity on values. Therefore $j(F) = \top$. When the guard fails, the output is zero, also true.

PA formalizes these constructor identities and proves by induction on a coded derivation that every derivable code has value true. Since code 1 has value false, PA proves consistency of S_{refl}. A PA proof of equivalence with its own consistency sentence would give a PA proof of Con(PA), contrary to the second incompleteness theorem if PA is consistent. □

The original witness head fails this valuation whenever $H \neq D$: its value is $j(\mathsf{E}(H,D)) = \bot$. Thus the proof does not accidentally prove consistency of the original calculus inside PA.

This is a barrier to one specific logical change, not a lower bound of 196 states. The accompanying Python program checks the exact numerical constructors, 384 head instances, 36 guarded gates, and all 16 Boolean instances of the propositional head in a small parameter box. These are regression checks; the universal proof is the argument above.

6.1 Why the first head also needs care

The first head supplies the system's explosion and negative-vacuity infrastructure. Replacing it by $\mathsf{U}(D,H) \to H$, or placing the sole copy of it inside a negative conjunction, cannot be justified by simply reusing the old classical conjunction-elimination proof.

A countervaluation sets the distinguished contradiction code to true, makes every equality atom false, interprets quantifiers as the identity, and makes some other membership atom false. Then all negative bundles are automatically true, the remaining positive heads are true, but $1 \to H$ can be false. The old explosion bootstrap has been lost. Details and noncircular safe alternatives are in `proof_notes/h1_bootstrap_constraints.md`.

The shorter fresh-label head $(\exists_H H) \to H$ is sound because label H is absent from decoded formula H, and it restores explosion. It does not immediately supply arbitrary fresh-variable vacuity or

the specialization used in the reverse arithmetic interpretation. No completeness theorem for that replacement was obtained, and its grammar screens were not competitive in any case.

7 New computational searches and their limits

7.1 Reassociation and the invalid 195-state merge

A rotation at native pair $(1362, 1361)$ changes an AX tree fragment from $(A, (B, C))$ to $((A, B), C)$. The new literal rows of 1361 match those of 1324, allowing a valid identical-row quotient of the altered table to 195 states. The source ordinal control flow can be preserved. However, the rotation has already changed the tapes on which the readers must be correct.

The row quotient is not the erroneous step: it preserves the altered table's behavior exactly. What fails is the preceding rotation's correspondence with the intended source program. The two underlying source roles have children (x_{21}, x_{23}) and (x_{21}, x_{55}). Their second children share physical reader 1355 by consulting prior tape history. After reassociation, the history class of x_{55} changes. The literally equal parent rows therefore do not implement equal source roles on the new tapes.

At native 1355 the intended interfaces are

Logical role	Read 0	Read 1
x_{55}	0 R 90	1 R 1312
x_{23}	0 R 12	1 R 1322

A second rotation repairs an overlong zero run but does not repair this reader. In that layout some required histories agree through six preceding tape cells and only differ at the seventh. The two-rotation literal candidate fails 93 of the 4,959 contracts. An *ideal, unimplemented* router choosing the intended interface passes all 4,959 contracts. That diagnostic isolates the obstacle; it is not a 195-state Turing machine.

A previously supplied alternative 196-state grammar gives analogous apparent 195-state reductions around reader 2037. The shortest history distinction found there uses three preceding bits but also introduces two scanner-contract failures. Other rotations repair the scanner while requiring four or five history bits, and one intermediate layout also breaks the left-entry convention of native 194. An ideal router alone is therefore not always sufficient.

For four layouts, a coupled C++ search enumerated all $780^2 = 608,400$ pairs of replacement actions at the shared reader, allowing recursive use of the changed rows. Each search checked 1,301 distinct local contexts, a 45-cell radius, and 240 literal steps. No survivor was found. Separate searches allowing one donated row elsewhere also failed within their stated bounds. These results do not exclude a longer excursion, a different invariant, or a repair changing several other rows.

7.2 The return scanner

The baseline native zero chain is

$$2 \to 3 \to 5 \to 9 \to 15 \to 23 \to 34 \to 75.$$

It distinguishes a return gap from a source address. Static counting of zeros before an existing one misses temporary bits introduced by decrement and return routines.

Deleting only state 34 causes 107 of the 4,959 universal branch checks to fail. Removing more scanner states does not repair the problem. In particular, the candidate deleting native 15, 23, and

34 halts from the blank tape after 699 transitions. This is an actual blank-run counterexample, stronger than failure of a local simulation contract.

A diagnostic extra scanner state repairs the two scanner failures in the one-rotation alternative layout when combined with the ideal router. Folding this extra state into the existing controls did not produce an accepted candidate. Neither a hypothetical router nor an added state is counted as a successful reduction.

A final layout screen asked whether all six-zero source addresses could first be shortened, allowing a smaller scanner. None of the initial CFG-preserving rotations removes source ordinal 0's leading six zeros. Two root rotations, $(84, 53, 0)$ then $(53, 310, 1)$, restore the source CFG and shorten that address, but change native 69's incoming history and fail the zero branch at source ordinal 11. This is a bounded failure report, not an impossibility theorem for shorter address layouts.

7.3 Engine folds and reachable source contexts

New depth-eight reverse-history analysis generated 13,273 partial-tape context cubes. Protected engine aliases were tested with bounded forward excursions and explicit branching on unknown tape cells. A separate screen examined 25 engine-state pairs sharing one branch, and another examined 4,232 pairs with independently synthesized branch actions. The latter tried 3,318,672 branch-action candidates and found none within its bounds. An append-zero register-tail variation was also screened, but it breaks the blank initializer and is not an accepted machine.

A zero/positive abstract interpretation from the exact clean source entry reached 274,880 abstract states and 3,810 of 3,812 source ordinals. At ordinal 3773 only the zero outcome occurs; ordinals 3774 and 3775 are absent. This allows three contracts to be omitted under that abstract invariant, leaving 4,956. A new restricted fold search tested 273,436 bounded local actions, found 1,202 locally interchangeable actions and 55 candidate state pairs, and rejected every concrete merged table on a retained contract. Merely pruning those two source occurrences gives no immediate state deletion because their decoder controls are shared extensively elsewhere.

Refining each register to the classes $\{0, 1, \geq 2\}$ reaches 4,455,507 abstract states and additionally excludes source ordinal 8; the decrement at ordinal 7 is zero-only. A $\{0, 1, 2, \geq 3\}$ abstraction reaches 32,947,398 states and gives the same three unreachable ordinals. Removing all five impossible contracts leaves 4,954. The corresponding bounded fold screen again produces 55 pairs and rejects them all. These computed closures require an abstract-transfer soundness proof before serving as a formal PA invariant; no new Lean certificate is claimed.

Two coupled searches then merge, respectively, native tail 62 or halt-parent 68 with each of 172 other controls, allowing both replacement rows to call the changed state recursively. Each screen tests 105,720,832 row pairs within a 20-cell radius and 240 transitions. Neither finds a local survivor. This does not exclude larger repairs or a different invariant.

The Boolean abstraction also finds every pair of logical registers interfering under the conservative rule that the unused partner must be zero whenever either register is accessed. This is not a proof against a more sophisticated relational register allocation.

7.4 Logical and grammar searches

The searches explored both source identities and logically equivalent axiom packages. Representative results are summarized below. A cost is a grammar/folding screening score unless a concrete table and its full validation are explicitly stated. A maximum matching of local folds need not be jointly realizable. None of these scores is a theorem of minimality.

Family	Result and limitation
Joint helper-selector MILP	Original 68-node AX grammar: optimal net saving six in the finite tested model, even allowing up to four new helpers; still 196 states.
Gate fusion of head 2	Logically recoverable with explicit treatment of failed guards; tested grammars did not improve the count.
Fusion into negative head 5	Head 1 retained for bootstrap; 30,720 logical variants screened. Best assembled candidate in the reported run has 198 states and passes 4,994 symbolic contracts.
Seven fusion helper MILPs	Each finite selector model solved optimally; no improvement beyond its existing folds.
Other transported words	Seven valid 197–199-state starting layouts; 34,491 physical rotation proposals through depth five found no accepted reduction.
Safe gate rearrangements	96 words screened; no net AX score below 62.
Single-site truth wrappers	53 words screened; best observed AX score 63.
Safe first/fifth-head weakenings	612 words screened; best observed net AX cost 64.
Exact zero-code padding	255 nontrivial selective expansions screened; no improvement in the tested dictionary model.
Alternative pairing macros	Bit-pairing changed the complete source DAG from 159 to 170 nodes in the implemented comparison.

An additional search explores valid physical rotations of the original table, including regions outside AX. It examined 19,930 distinct proposals through six levels, retaining up to 100 valid layouts per frontier. A total of 2,961 layouts passed the source-CFG, address, and pure-decoder screens; none exposed an additional candidate fold under this model. Exact bounds are recorded in `search_root/global_rotations.result.json`. This is a bounded search, not a minimality proof.

The correctness of safe logical rewrites must be established before state counting. For example, conjoining positive heads into the negative witness head is legitimate after the unchanged first head supplies classical reasoning. Conjoining the *only* bootstrap head into such a bundle is not justified by the same argument.

7.5 Register reassignment: a new 196-state control example

Changing register locations preserves the abstract source program only after both runtime and startup have been checked. Retargeting the eleven non-engine entry rows into the register-selector states gave three permutations with all 4,959 runtime contracts passing under the changed assignment. Two failed to reach the required clean frame within one million blank-tape transitions.

The assignment

$$(6, 2, 0, 3, 4, 7, 8, 5, 1)$$

exchanges the physical positions of logical registers A and D relative to the baseline $(6, 2, 0, 4, 3, 7, 8, 5, 1)$. It passes all 4,959 contracts and reaches the exact clean frame after 58,122 literal transitions. Its table and portable verifier are included. This is a new executable-checker-validated 196-state variant, not a sub-196 result and not a new Lean proof.

Within the tested one-step contextual-selector model, these physical permutations cannot add a fold. The eleven changed rows have action form $(0, R, \mathrm{SK}[p])$ on read-one, so none is a pure bit-preserving provider. No unchanged engine row has the same action form. A bijection of selector targets therefore preserves the relevant action-equality, provider, and incoming-context relations. This restricted invariance says nothing about more general changes to the engine or decoder.

8 What ConciseIPRA contributes

The inspected ConciseIPRA material presents compact theories related by interpretation to intuitionistic primitive recursive arithmetic, with equation induction. Replacing the existing proof search by a checker for such a theory would change the target consistency statement unless a separate PA-strength argument is supplied.

The useful transferable ideas are structural: compact witness management, dispatch driven by an available witness, and shifting work into coded syntax. They are not a license to replace full arithmetic induction by a weaker rule while retaining the label Con(PA).

One research direction is to extend a compact primitive recursive arithmetic core to induction for all coded formulas, then connect the resulting intuitionistic theory to HA and PA by explicit proof translations. This would require a complete checker, a binary-tape compiler, and an internal equivalence proof. A short high-level Python checker is not evidence of a small blank-tape binary Turing machine, and normalization is not a free machine primitive. No such new construction is asserted here.

9 Reproduction, artifacts, and remaining work

9.1 Archive layout

`report/`
This TeX source and compiled PDF.
`comparison/`
Eleven literal tables, source site lists, state conjugacies, exact comparison code, source manifest, fresh output, and inherited license chain.
`proof_notes/`
Detailed reverse-consistency strategy, reflexivity obstruction, bootstrap constraints, and bounded regression checker.
`search_root/`, `research196/`, `research208/`, `research_ipra/`
New search code, diagnostic candidates, and receipts. Candidate names containing 195 do not imply validity; consult `STATUS.json`.
`support/`
Clearly identified source snapshots, previously supplied checker/grammar infrastructure, alternative 196-state layout, and inherited physical Lean material.
`run_checks.py`
Offline entry point for the principal finite checks and literal startup.

The archive manifest hashes every delivered file. Main checks use Python's standard library. The optional MILP search needs NumPy and SciPy; local excursion screens need a C++17 compiler. The report builds with pdfLaTeX. Full Lean reproduction requires the toolchain specified by the included source snapshot and its dependencies; the quick checks do not claim to rebuild that development.

9.2 What would complete the requested result

A successful continuation must supply all of the following:

1. A literal table with at most 195 working states in the same binary blank-tape model.
2. Exact startup into a proved invariant, or a complete direct blank-run simulation.
3. Correct primitive/source behavior on the necessary invariant, including changed history readers, scanner behavior, and all affected register excursions.
4. A PA-formalizable proof of both consistency implications. Reusing the 196-state result requires an internally justified halting equivalence, not merely matching bounded runs.

No search result here proves that this is impossible. The completed comparison rules out a simple additive use of the existing U208 savings. The reflexivity theorem rules out one specific reduction of logical strength. The rejected 195-state layouts identify concrete obligations for more sophisticated history encodings. The construction requested by the user remains open in this report.

References

[1] James Chang, *BusyBeaver*, revision `407cdc356f6b586635e71ac92be394255f4bfea4`. Literal table, source, logical development, and arithmetic endpoints.
https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4

[2] James Chang, *BusyBeaver*, requested 209-state revision `69d98241a721ead20b4f0b17df9062950b40fbfe`.
https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe

[3] Vladimir Reshetnikov, *ProveIt*, incoming research at revision `f7d2482e35cf2f8f3bd03b5a82387ad999243548`. Includes construction packages, comparison reports, and the earlier unsuccessful below-196 archive.
https://github.com/VladimirReshetnikov/ProveIt/tree/f7d2482e35cf2f8f3bd03b5a82387ad999243548/docs/incoming

[4] Vladimir Reshetnikov, *ConciseIPRA*, README blob `5526b516e20d61d89c1a1575744d35742f669efd`, within the same pinned ProveIt revision.
https://github.com/VladimirReshetnikov/ProveIt/tree/f7d2482e35cf2f8f3bd03b5a82387ad999243548/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA